NAME: KEERTHIKA S **ROLL NO: 231901024**

TITLE: EXP NO.: 9 MONITORING AND LOGGING FABRIC NETWORKS

1. AIM

To monitor the Hyperledger Fabric network by inspecting Docker containers, viewing peer and orderer logs, checking network status, monitoring resource utilization, and analyzing blockchain transaction logs for troubleshooting and performance analysis.

2. TOOLS / SOFTWARE REQUIRED

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

3. HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

4. THEORY

Monitoring and logging are essential for maintaining a healthy Hyperledger Fabric network. Administrators use monitoring tools and log files to verify that peer nodes, orderers, Certificate Authorities, and chaincodes are functioning correctly. Logs help identify: • Network startup failures • Chaincode deployment issues • Transaction processing errors • Peer communication failures • Ordering service problems • Certificate authentication errors • Performance bottlenecks Docker provides built-in

commands for monitoring containers and viewing logs, making it easier to troubleshoot Hyperledger Fabric networks.

5. PROCEDURE

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

```
./network.sh up createChannel -ca
```

Step 3: Verify Running Containers

Display all active Docker containers.

```
docker ps
```

Step 4: Display Docker Container Statistics

Monitor CPU and memory usage of Fabric containers.

```
docker stats
```

Step 5: View Peer Log

Display the log of Peer Organization 1.

```
docker logs peer0.org1.example.com
```

Step 6: View Orderer Log

```
docker logs orderer.example.com
```

Step 7: View Certificate Authority Log

```
docker logs ca_org1
```

Step 8: Verify Network Channel

```
peer channel list
```

Step 9: Query Installed Chaincode

```
peer lifecycle chaincode queryinstalled
```

Step 10: Monitor Blockchain Transactions

Query all assets stored in the ledger.

```
peer chaincode query
-C mychannel
-n basic
-c '{"Args":["GetAllAssets"]}'
```

Step 11: View Recent Docker Events

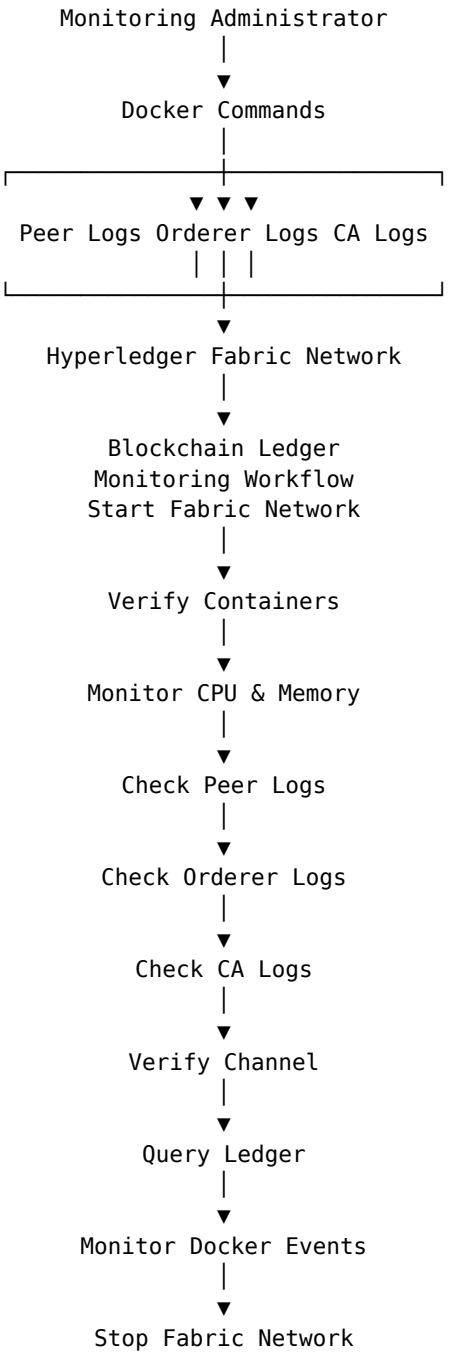
Monitor Docker events generated by the Fabric network.

```
docker events --since 10m
```

Step 12: Stop the Fabric Network

```
./network.sh down
```

6. ARCHITECTURE DIAGRAM



7. SUMMARY OF KEY COMMANDS

Purpose	Command
Navigate	`cd ~/fabric-dev/fabric-samples/test-network`
Start network	`./network.sh up createChannel -ca`
Check containers	`docker ps`
Monitor resources	`docker stats`
View logs	`docker logs peer0.org1.example.com`
View logs	`docker logs orderer.example.com`
View logs	`docker logs ca_org1`
Verify channel	`peer channel list`
Query ledger	`peer lifecycle chaincode queryinstalled`
Query ledger	`peer chaincode query`
Command	`docker events --since 10m`
Stop network	`./network.sh down`

8. OUTPUT

The terminal output below is rendered from the output blocks supplied in the source experiment.

Output 1

```
Experiment 9 - Fabric Output

$ Fabric Network Started Successfully
Docker Containers Running
Peer Logs Verified
Orderer Logs Verified
Certificate Authority Running
Channel Verified Successfully
Chaincode Verified Successfully
Network Monitoring Completed Successfully
```

Fig. 9.1 – Fabric output corresponding to the source experiment output.

9. RESULT

The Hyperledger Fabric network was successfully monitored using Docker utilities and Fabric CLI commands. Peer nodes, ordering service, Certificate Authorities, and blockchain transactions were verified through log analysis and monitoring tools. The experiment demonstrat ed effective monitoring and troubleshooting techniques for maintaining a reliable Hyperledger Fabric network.